ARCHITECTURAL SPECIFICATION & TECHNICAL SYNOPSIS

AI Model Compiler & Optimization Engine

A research-oriented deep learning compiler engineered to ingest PyTorch models into a custom, project-owned Intermediate Representation (IR), enforce graph-wide structural invariants, apply algebraic and topological optimization passes, perform precision reduction, and verify numerical equivalence against transparent reference execution.

`Package: ai-model-compiler v0.1.0` `Runtime: Python 3.10+ / PyTorch CPU`

`Target: Inference Graph Optimization` `Graph Model: Custom Project-Owned DAG IR`

`UI: Tkinter Interactive Workbench`

1. Executive Summary & Core Motivation

Modern deep learning frameworks prioritize developer agility and dynamic training flexibility over execution efficiency. Deploying trained neural networks onto resource-constrained or specialized hardware requires a compilation stage: lowering high-level framework graphs into hardware-efficient intermediate representations, eliminating redundancies, fusing operator sequences, and reducing precision.

The **AI Model Compiler & Optimization Engine** is built as an end-to-end prototype that explores the core mechanics of a neural network compiler. Unlike projects that simply wrap runtime engines like ONNX Runtime, TensorRT, or TVM, this engine maintains **complete ownership over its Intermediate Representation (IR)**, graph traversal algorithms, pass manager lifecycle, and transformation passes. PyTorch's `torch.fx` is used strictly at the ingestion boundary for symbolic extraction, and PyTorch functional kernels serve as a pure reference execution baseline.

CORE DESIGN PRINCIPLES

- **IR Independence:** Optimization rules and graph transformations mutate only the project's internal Graph/Node/Tensor IR.
- **Strict Invariant Verification:** The graph structure, producer/consumer links, topological ordering, and acyclic properties are validated after ingestion and after *every single compiler pass*.
- **Isolated Verification:** Graph transformation logic is isolated from kernel implementation bugs by executing the IR against transparent reference kernels and comparing outputs within strict mathematical tolerances.
- **Empirical Rigor:** Performance claims require repeated timing distributions (mean, median, p95) with warmups and environment metadata, tracking neutral and negative results.

2. End-to-End Compiler Pipeline

The compilation lifecycle follows a linear pipeline transitioning through six distinct phases:



3. Custom Intermediate Representation (IR) Architecture

Located in `compiler/ir/`, the custom intermediate representation structures neural network computations as a clean Directed Acyclic Graph (DAG) with explicit dataflow contracts:

Graph Container (graph.py)

- **Named Registries:** Stores `nodes: dict[str, Node]` and `tensors: dict[str, Tensor]`.
- **Boundaries:** Declares ordered `inputs: list[str]` and `outputs: list[str]`.
- **Topological Order:** Implements Kahn's algorithm with incoming dependency counters. Detects and reports graph cycles immediately.
- **Live Analysis:** Implements `reachable_nodes()` via reverse-traversal from outputs.

Node & Tensor Abstractions (node.py, tensor.py)

- **Node:** Represents an atomic operation (e.g. `Linear`, `Conv2d`). Holds input IDs, output IDs, attributes, kernel parameters (stride, padding, dilation), and compiler metadata.
- **Tensor:** Value-centric object storing static multidimensional shape (tuple), dtype string, producer node ID, set of consumer node IDs, constant tensor value, and quantization scale.

STRUCTURAL INVARIANTS ENFORCED BY GRAPH.VALIDATE()

1. Every node input and output ID must exist in the tensor registry.
2. Each computed tensor has exactly one producer; graph inputs and constant weights have `producer=None`.
3. Producer and consumer relationships are strictly symmetric (`node ∈ tensor.consumers ⇔ tensor ∈ node.inputs`).
4. Declared graph boundaries (inputs and outputs) must exist and be flagged accordingly.
5. The computational graph must remain strictly acyclic.

4. PyTorch Frontend & Operator Ingestion

The ingestion subsystem (`compiler/frontend/pytorch_importer.py`) bridges high-level PyTorch models into the custom IR:

- **Symbolic Extraction:** Calls `torch.fx.symbolic_trace(model.eval())` to construct a symbolic execution graph.
- **Shape Propagation:** Leverages PyTorch's `ShapeProp` pass using example inputs to annotate every intermediate node with output tensor dimensions and data types.
- **Attribute & Parameter Mapping:** Constant weights and biases from PyTorch modules are extracted, detached, converted into standalone constant `Tensor` objects, and wired as explicit inputs to consumer nodes.
- **Operator Registry:** Supports both module-level and function-level PyTorch representations:

IR Operator	PyTorch Source	Captured Parameters / Attributes	Status
`Linear`	`nn.Linear`, `F.linear`	`in_features`, `out_features`, weights, bias	Implemented
`Conv2d`	`nn.Conv2d`	`stride`, `padding`, `dilation`, `groups`, weights, bias	Implemented
`BatchNorm2d`	`nn.BatchNorm2d`	`eps`, `momentum`, `running_mean`, `running_var`, γ, β	Implemented
`MaxPool2d` / `AvgPool2d`	`nn.MaxPool2d`, `nn.AvgPool2d`	`kernel_size`, `stride`, `padding`, `dilation`, `ceil_mode`	Implemented
`Flatten`	`nn.Flatten`	`start_dim`, `end_dim`	Implemented
`ReLU`	`nn.ReLU`, `torch.relu`	None (elementwise non-linearity)	Implemented
`Add` / `Mul`	`torch.add`, `torch.mul`, `+`, `*`	Broadcastable elementwise operands	Implemented

Diagnostic Discipline: When tracing encounters unsupported modules (e.g. `nn.Sigmoid`, RNNs, MultiheadAttention) or dynamic control flow, the compiler halts immediately with a clear `NotImplementedError("Unsupported module: <Name>")` rather than silently producing an incomplete graph.

5. Optimization Engine & Compiler Passes

The optimization engine (`compiler/optimizer/`) operates through an extensible `PassManager`. Each pass receives the graph, applies topological rewrites, updates consumer sets, and triggers graph-wide validation.

Constant Folding (constant_folding.py)

Identifies operations where all input tensors have static constant values (such as pre-calculated scaling factors or bias offsets). Evaluates the arithmetic at compile time, attaches the resulting values to the output tensor, marks the tensor as constant, discards the parent node, and updates downstream consumer links.

Dead-Node Elimination (dead_node_elimination.py)

Computes the transitive closure of all nodes contributing to the graph's declared outputs using reverse graph traversal. Any unreachable node (e.g. orphaned intermediate representations or detached branches) is purged from the node registry along with its dangling intermediate outputs.

Redundant Operation Elimination (redundant_ops.py)

Searches for scalar identity arithmetic: `x + 0` or `x * 1`. When unambiguous broadcasting is proven, the pass bypasses the operation by rewiring all downstream consumer nodes directly to the dynamic input operand, removing unnecessary kernel launches and memory traffic.

Conv-BatchNorm-ReLU Fusion (operator_fusion.py)

Fuses sequential `Conv2d → BatchNorm2d → ReLU` pipelines into a unified `FusedConvBatchNormReLU` node. By folding the batch normalization parameters directly into the convolution weights and biases, inference execution completely bypasses the separate BN kernel invocation and memory roundtrip.

Conv-BatchNorm Parameter Folding Derivation (Inference Mode):

Given convolution weight W, bias b, and BatchNorm parameters (μ, σ^2, γ, β, ε):

- **Scale Factor:** $s = \gamma / \sqrt{(\sigma^2 + \varepsilon)}$
- **Fused Convolution Weight:** $W_{\text{fused}} = W \times \text{reshape}(s, [-1, 1, 1, 1])$
- **Fused Convolution Bias:** $b_{\text{fused}} = (b - \mu) \times s + \beta$

The three individual nodes and intermediate feature maps are eliminated from the IR and replaced by a single compound node pointing to W_{fused} and b_{fused}, preserving exact numerical output semantics.

6. Quantization & Precision Engine

The precision subsystem (`compiler/quantization/precision.py`) implements low-precision representation techniques to reduce memory footprint and latency:

Symmetric Weight-Only INT8

Quantizes floating-point weight tensors of `Linear`, `Conv2d`, and fused layers into signed 8-bit integers:

$scale = \max(|W|) / 127.0$

$W_{\text{int8}} = \text{clamp}([\ W / scale\ ], -127, 127)$

Stores the resulting `int8` tensor along with its float32 dequantization scale on the custom `Tensor` object.

Activation Calibration & Range Estimation

Feeds representative input batches through the reference runtime with `return_intermediates=True`. Observes the empirical dynamic range $[\min(A), \max(A)]$ for every intermediate activation tensor to compute symmetric calibration scale factors:

$scale_{\text{act}} = \max(|\min|, |\max|) / 127.0$

Numerical Error Quantification (numerical_error)

Ensures that optimization and quantization passes do not corrupt computational accuracy by computing absolute and relative discrepancies against the original PyTorch model:

- **Maximum Absolute Error:** $\max |Y_{\text{candidate}} - Y_{\text{reference}}|$
- **Mean Absolute Error (MAE):** $(1 / N) \sum |Y_{\text{candidate}} - Y_{\text{reference}}|$
- **Maximum Relative Error:** $\max [\ |Y_{\text{candidate}} - Y_{\text{reference}}| / \max(|Y_{\text{reference}}|, \varepsilon)\ ]$

7. Backend Execution & Empirical Benchmarking

Reference CPU Backend (compiler/backend/reference.py)

The reference execution backend is an interpreter that iterates through the graph in topological order. For each node, it resolves input tensor values (unpacking dynamically supplied model inputs, constant parameters, and applying on-the-fly dequantization $W_{\text{fp32}} = W_{\text{int8}} \times \textit{scale}$), dispatches to the corresponding PyTorch functional kernel, and maps the output back to the value registry. This decouples compiler validation from low-level C++/assembly kernel development.

Repeatable Latency & Throughput Runner (benchmarks/runner.py)

The benchmarking harness strictly isolates warmup effects from measurement runs:

- **Configurable Warmup:** Runs *N* unmeasured warmup iterations to prime instruction and CPU caches.
- **Distributional Metrics:** Executes *M* timed iterations using `time.perf_counter()`, calculating mean latency, median latency, and 95th-percentile (p95) latency in milliseconds.
- **Throughput Calculation:** Reports processed samples per second (BatchSize / (MeanLatency / 1000)).
- **Reproducibility Metadata:** Captures OS platform, Python version, PyTorch version, batch size, and execution hardware.
- **Persistence:** Exports structured results to both JSON and CSV files (`results/ablation.csv`).

8. Interactive Desktop Workbench (GUI)

In addition to the programmatic and CLI interfaces, the project includes an interactive Windows-compatible desktop workbench (`compiler/gui.py`) implemented in Tkinter:

- **Workload Selection:** Quickly toggle between predefined models (e.g. multi-layer perceptron `MLP demo`, convolutional network `CNN demo`).
- **Precision Control:** Select between FP32, FP16, and INT8 precision modes.
- **Interactive Pass Configuration:** Checkboxes allow users to selectively enable or disable individual optimization passes (Constant folding, Dead-node elimination, Redundant op removal, Conv-BN-ReLU fusion) to observe isolated effects.
- **Live Telemetry:** Renders compilation status, pass transformation statistics, reference vs compiled numerical error, and latency distributions directly in the UI console.
- **IR Export:** One-click JSON export of the compiled custom IR graph for visualization and archiving.

9. Repository Layout & Verification Matrix

Directory / File	Core Responsibility	Key Outputs / Tests
`compiler/frontend/`	PyTorch FX symbolic graph tracing, shape propagation, operator registry	`pytorch_importer.py`
`compiler/ir/`	Graph, Node, Tensor data structures, validation rules, JSON serialization	`graph.py` , `validation.py`
`compiler/optimizer/`	PassManager orchestration; constant folding; DNE; redundant ops; fusion	4 optimization passes
`compiler/quantization/`	Symmetric INT8 weight quantization, activation calibration, numerical error	`precision.py`
`compiler/backend/`	Topological graph interpreter using reference PyTorch kernels	`reference.py`
`compiler/gui.py`	Interactive desktop compilation workbench with live controls	Tkinter UI
`benchmarks/`	Latency/throughput runner, torchvision evaluation harness, CSV/JSON persistence	`runner.py` , `workloads.py`
`visualization/`	Graphviz DOT generator (no Graphviz runtime dependency), Matplotlib plotter	`dot.py` , `benchmarks.py`
`tests/`	Automated unit and integration test suite across the complete pipeline	6 test suites (passing)

10. Automated Test Suite Coverage

- `tests/test_vertical_slice.py` : Validates end-to-end MLP import, reference execution equivalence, and unsupported module diagnostics.
- `tests/test_optimizations.py` : Verifies constant folding of Add/Mul, dead-node elimination of unreachable subgraphs, and redundant scalar zero/one elimination.
- `tests/test_fusion_and_precision.py` : Tests Conv-BN-ReLU fusion numerical close-match, FP16 downcasting, INT8 weight quantization tolerances, and activation range calibration.
- `tests/test_gui_pipeline.py` : Asserts that programmatic calls from the GUI workbench produce valid, finite, and numerically accurate compiled outputs.
- `tests/test_persistence.py` : Checks JSON serialization round-tripping and CSV benchmark record parsing.
- `tests/test_tools.py` : Tests DOT visualization string generation and workload evaluation reporting.

11. Limitations & Planned Research Roadmap

As documented in `docs/PROJECT_BLUEPRINT.md`, the current implementation is a robust vertical slice with specific planned extensions:

Current Scope Boundaries

- **Inference Only:** Training semantics, backwards autograd graphs, and optimizer states are intentionally unsupported.
- **Static Graphs:** Dynamic shapes, data-dependent control flow (if-conditions, while-loops), and non-tensor FX targets are rejected.
- **CPU Reference Execution:** Kernel execution is mediated by PyTorch functional operations rather than a custom LLVM/C++ backend.

Next Development Milestones

- **Workload Expansion:** Expand operator coverage to support full, unmodified ResNet-18 and MobileNet-v3 graphs (residual skip additions, global pooling).
- **Comparative Benchmarks:** Add ONNX Runtime export and baseline comparison to evaluate custom IR performance against industry runtimes.
- **Native CPU Lowering:** Implement a lightweight C/NumPy lowering path to eliminate the PyTorch execution dependency entirely.
- **Dataset Accuracy Study:** Measure real-world accuracy degradation across CIFAR-10 / ImageNet for fused and quantized models.

CONCLUSION & RESEARCH SIGNIFICANCE

The project provides a clean, mathematically verified, and modular prototype of an AI compiler. By maintaining strict IR invariants, formal pass verification, transparent reference execution, and automated ablation persistence, it establishes a solid architectural foundation for deep learning compiler research and optimization experimentation.